

HotShot: Let the Parlays Begin!

Final Report

Field	Information
Student	Weichen Hung
Project	HotShot Sportsbook Web Application
Class / Semester	Senior Project, Spring 2026
Primary Technologies	React, Node.js, PostgreSQL, The Odds API
Submission Date	May 2026

This report summarizes the goals, design, implementation, challenges, and future improvements for the HotShot senior project.

Abstract

HotShot is a sportsbook-style web application designed to demonstrate interactive sports betting concepts in a safe points-based environment. The project allows users to create accounts, log in, browse NBA, NHL, and MLB matchups, place single-game moneyline bets, build parlays, view betting history, and compare user performance through a leaderboard. The app also includes sport-specific visual design, background images, market charts, locked and resolved games, and a scouting panel that summarizes matchup trends from stored team profiles and recent form data.

The project began as a frontend-focused prototype and grew into a full-stack MVP. A Node.js backend and PostgreSQL database were added so users, sessions, games, bets, parlays, and leaderboard results could persist beyond a single browser session. The app also includes partial integration with The Odds API to pull real sports odds and recent scores when available. By the final stage, HotShot demonstrated the main requirements of an MVP: persistent accounts, database-backed betting activity, API-connected sports data, visible user outcomes, and a presentation-ready interface.

Project Overview

The goal of HotShot was to build a web app that felt like a live sports market while staying appropriate for a school project. Instead of using real money, users receive points and can risk those points on sports outcomes. This keeps the experience engaging while making it clear that the app is a simulator and not an actual gambling product.

The app focuses on four main user workflows: account creation and login, game browsing, bet placement, and performance tracking. Users can search teams, view matchups by sport, inspect market charts, place straight bets, combine games into parlays, and then see their results reflected in their history and leaderboard standing.

Area	Final Implementation
Frontend	React single-page application with game views, login, parlays, history, leaderboard, and responsive styling.
Backend	Node.js API with account, session, game, bet, parlay, settlement, leaderboard, and API sync routes.
Database	PostgreSQL tables for users, sessions, sports, teams, games, bets, parlays, and parlay legs.
External Data	The Odds API support for odds and recently completed scores.
Demo Safety	Points-based betting only; no real money is used.

Design and Implementation

Frontend Design

The frontend was built in React. I chose React because it allowed the app to update quickly as users switched sports, searched teams, selected games, placed bets, and watched market results change. The app uses a single-page layout with navigation for games, parlays, history, and leaderboard. Each section was designed to be understandable during a live presentation, with clear status labels for live, upcoming, locked, and resolved games.

A major design goal was making the app feel engaging instead of looking like a plain database form. The NBA, NHL, and MLB sections use sport-specific background images. The login page uses the basketball background, while parlays and leaderboard use hockey and baseball themes. This visual layer helped the project feel more complete and made the demo easier to follow.

Backend and Database

The backend was added to make the app meet MVP expectations. Instead of only storing information in browser state, HotShot now stores important user information in PostgreSQL. When a user creates an account, the backend saves the username, password hash, points, wins, and losses. When the user places a bet or parlay, the backend saves that ticket and subtracts the wager from the user's points. When a result is settled, the backend updates the ticket result and the user's leaderboard totals.

This database layer was one of the most important improvements because it made the app persistent. A user can log out, log back in, or use a different account and still see database-backed information. The leaderboard now reads real PostgreSQL user records instead of showing fake leaderboard entries.

Betting and Parlay Rules

The betting rules were refined throughout the project. Final games and locked games cannot be bet on. Games more than the active betting window are visible for scouting but locked. A user cannot place a straight bet and then reuse the same game in a parlay. The backend enforces these rules so they cannot be bypassed by only changing the frontend.

Parlays were also improved near the end of the project. At first, parlays depended too much on a browser timer, which could leave a parlay pending if the page refreshed. The final version includes backend settlement logic so pending parlays can resolve and update PostgreSQL, points, wins, losses, and the leaderboard.

API Integration

HotShot includes support for The Odds API. The backend can sync odds and recent scores for NBA, NHL, and MLB games when an API key is configured. Some games are still included as a stable starter slate so the app remains reliable for presentation, but the project demonstrates how outside sports data can be brought into the backend and used by the frontend.

What Went Right

- The project became a working full-stack MVP instead of remaining only a frontend prototype.
- User accounts, betting history, parlay activity, and leaderboard totals are stored in PostgreSQL.
- The app has a clear and engaging user experience with sports-specific pages and visual assets.
- Important betting rules are enforced by both the frontend and backend.
- The Odds API integration shows how the project can connect to real sports data.
- The project is presentation-ready because it includes enough current and upcoming games to demonstrate multiple workflows.

Challenges and What Went Wrong

The biggest challenge was moving from a local frontend demo to a full-stack application. At first, the app could show bets and points in the browser, but that was not enough for the MVP because the data needed to persist in a database. Adding the backend and PostgreSQL changed the project from a visual prototype into a real application with user records and stored transactions.

Another challenge was handling edge cases. For example, the app needed to prevent duplicate usernames, stop users from betting the same game twice, lock games that should not be bet on, mark resolved markets as closed, and update the leaderboard after results. Each of these issues made the app more complete because they forced the project to behave like a real system instead of only looking correct during a simple demo.

Public hosting was also more complicated than expected. Hosting the frontend alone is not enough because a full-stack MVP also needs a hosted backend and hosted database. This helped me understand the difference between running an app locally and deploying a complete web application that other people can use.

What I Learned

This project taught me how different parts of a web application work together. I learned that a React frontend is only one piece of the system. For users, bets, and leaderboard points to persist, the app also needs backend routes, database tables, authentication/session handling, and careful communication between the frontend and backend.

I also learned the importance of testing real user flows instead of only checking individual features. Some problems only became clear when I created multiple accounts, placed bets, checked the leaderboard, refreshed the page, or tried to reuse the same game in a parlay. Testing the complete flow made the project stronger.

Finally, I learned how much deployment matters. A project can work on localhost but still need extra setup before it works publicly. Understanding environment variables, backend URLs,

database connection strings, API keys, and build settings was an important part of turning HotShot into a complete MVP.

Design Specification Changes

Several changes were made after the original design direction. The most important change was adding a backend and PostgreSQL database. The original version focused more heavily on frontend interactions, but the final version includes persistent storage for users, bets, parlays, and leaderboard data.

The game data approach also changed. The app now supports The Odds API for odds and recent results, while still keeping a stable starter slate of games so the presentation does not depend entirely on live API availability. The betting window was adjusted to three days so the app can show upcoming games while locking games that are too far away. Team lists were also updated as playoff outcomes changed, and resolved/eliminated teams were handled with clearer messages.

Future Improvements

- Deploy the backend and PostgreSQL database publicly so the hosted frontend uses the same full-stack behavior as the local version.
- Expand API syncing so more games, odds, and final scores update automatically from real sports data.
- Add more bet types such as totals, spreads, and player props.
- Improve account security with stronger password hashing and password reset support.
- Add an admin tool for managing games, locked markets, and presentation data without editing code.
- Improve accessibility with stronger labels, keyboard navigation checks, and more formal screen-reader support.

Conclusion

Overall, HotShot successfully met the main goal of becoming an interactive sports betting simulator with persistent user data and a working full-stack structure. The project includes a polished React frontend, a Node.js backend, PostgreSQL storage, API integration, account rules, betting rules, parlay logic, history, and a real leaderboard. The final version is stronger than the original prototype because it does not only look like an app; it stores and updates real application data.

If I were to continue the project, my next priority would be public deployment of the backend and database, followed by deeper API automation and more bet types. Even with those possible improvements, the current version demonstrates the core MVP clearly and shows what I learned about frontend development, backend development, database design, API integration,

and full-stack project planning.

Appendix: Interview Talking Points

Question	Strong Answer
Where is data stored?	User accounts, bets, parlays, games, and leaderboard totals are stored in PostgreSQL through the Node.js backend.
Where do games and odds come from?	The backend supports The Odds API for odds and recent scores, with a stable starter slate for reliable demo coverage.
What is the scouting feature?	It is a rules-based matchup assistant built from stored team profiles, recent form summaries, and market movement.
What happens when a user loses all points?	The backend records when the user reaches zero. After 24 hours, the user can receive 100 points back.
Can users duplicate accounts?	No. The backend blocks duplicate usernames, including case-only duplicates.